OOP through Java Lab – Mini Project Guidelines

1 Objective of the Mini Project

The purpose of this mini project is to demonstrate your understanding and practical application of Object-Oriented Programming concepts using Java. Your project must clearly show proper class design, object interaction, and use of core OOP principles learned during the course.

This is not just about making a working program. It is about showing correct OOP design and implementation.

2 Team Size

- Team size: **1–3 students**
- All members must understand the full code
- Individual viva will be conducted

3 Mandatory OOP Requirements (Must Be Implemented)

Your project **must include all** of the following:

Class Design

- Minimum **6 user-defined classes**
- Classes must represent real entities of the system
- Avoid putting all logic inside one class

Encapsulation

- All data members must be **private**
- Use getters and setters
- Include validation inside setters where required

Inheritance

- At least **one inheritance hierarchy**
- Must include method overriding
- Proper use of `super` keyword where needed

Polymorphism

- Runtime polymorphism must be demonstrated
- Use base class reference pointing to child object
- Overridden methods must be executed through parent reference

Interface

- At least **one interface**
- Must be implemented by two or more classes

Abstract Class

- At least **one abstract class**
- Must contain at least one abstract method
- Subclasses must implement it

Constructor Usage

Must include:

- Default constructor
- Parameterized constructor
- Constructor overloading

Method Overloading

- At least one overloaded method required

Collections Framework

Use at least **two collection types**, such as:

- ArrayList

- HashMap
- HashSet
- TreeSet
- Queue

Do not rely only on arrays.

Exception Handling

- Use try–catch blocks
- Create at least **one user-defined exception**
- Handle invalid inputs properly

File Handling

Project must include at least one:

- File read/write
- Object serialization
- Text or CSV storage

Programs that keep all data only in memory will not be accepted.

Packages

- Use at least **two custom packages**
- Organize classes logically

Example:

```
model
service
util
ui
```

4 User Interface Requirement

Your project must provide either:

- Menu-driven console interface

  or
- GUI using Swing / JavaFX

Program must be interactive.

5 Code Quality Rules

- Use meaningful class and method names
- Follow proper indentation
- Add comments for important logic
- Avoid unused variables and dead code
- Avoid excessive static usage for core logic
- No copy-paste coding from internet sources without understanding

6 Documentation to Submit

Each team must submit:

Project Report (PDF)

Include:

- Problem statement
- Features list
- OOP concepts used
- Class diagram (UML)
- Package structure
- Sample outputs

- Team member contribution

Source Code

- Proper folder structure
- Packages included
- Must compile and run

7 Demonstration Requirements

During demo and viva you must be able to:

- Explain class design
- Explain inheritance and interfaces used
- Show polymorphism in execution
- Explain exception handling
- Modify small parts of code if asked
- Answer concept-based questions

8 Academic Integrity Rules

- Each team must choose a different problem or clearly different design
- Copying code from other teams = zero marks
- Copying from internet without understanding = rejected in viva
- Similar code structure across teams will be penalized